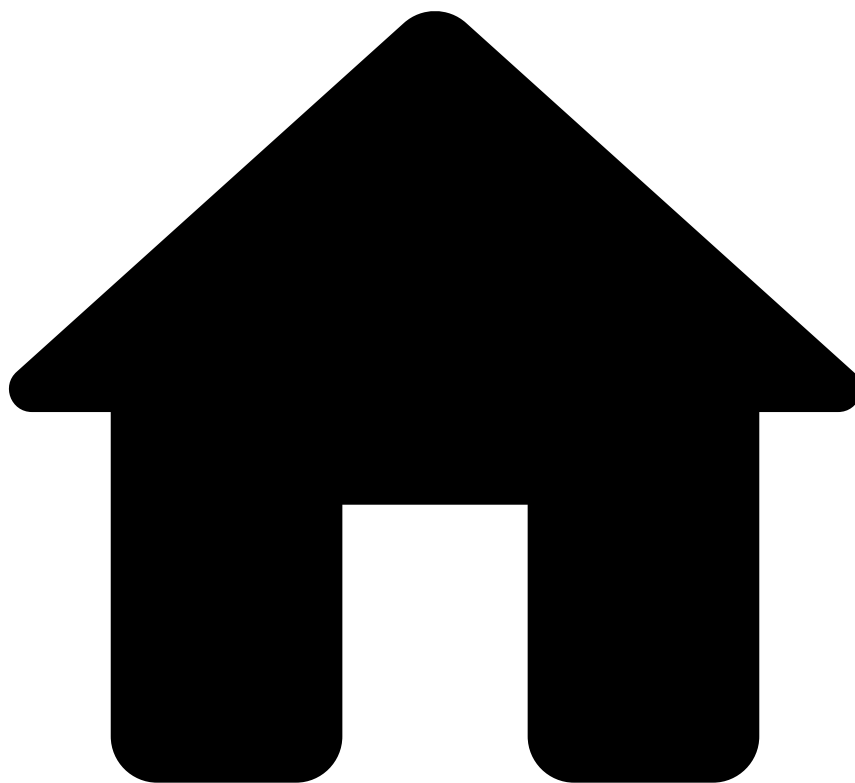


•



- [Core Concepts](#)
- [Data Science Loop](#)
- The Data Science Framework (DSF)

On this page

## The Data Science Framework (DSF)

Was this page helpful? 

The Data Science Framework (DSF), also known as the Data Science Studio, is a dedicated environment used in risk assessment strategies for building, testing, and evaluating rules and models according to the [Data Science Loop](#).

Its main capabilities include:

- **Data preparation:** Read production data or ingest historical data, clean it, and analyze its quality and statistics to ensure it fits your schema. This is crucial to work with Rules, Models and/or ad-hoc analysis.
- **Feature engineering and plan execution:** Create and test new features using Plans, which can then be deployed to production. Plans allow the application of generic data computations, typically to transform it or extend it to a new file output.

- **Model and rule development:** Train Machine Learning (ML) models and create rule sets, and evaluate their performance against historical data to identify if further improvements are needed.
- **Ad-hoc analysis:** Study and request ad-hoc reports or analysis via the DSF.
- **Sandbox Evaluations:** Test a sequence of a Model and Rule project in the Data Science application.

## What are Data Science jobs?

Data Science (DS) jobs are jobs initiated in the DSF environment, either via Pulse or JupyterLab, and processed through the Pulse application. While Pulse manages the job lifecycle, the heavy computational workload is shifted to the cluster, which allocates **executors** (i.e. processing units) throughout its **task nodes** (i.e. the actual machines that compose the cluster) to perform the computations as a Spark job.

Refer to [Spark's documentation](#) to further understand its core functionality.

The resources allocated to each job are guided by the job configuration and workload, and bounded by the cluster configuration (i.e. size and number of task nodes, as well as the executors per node).

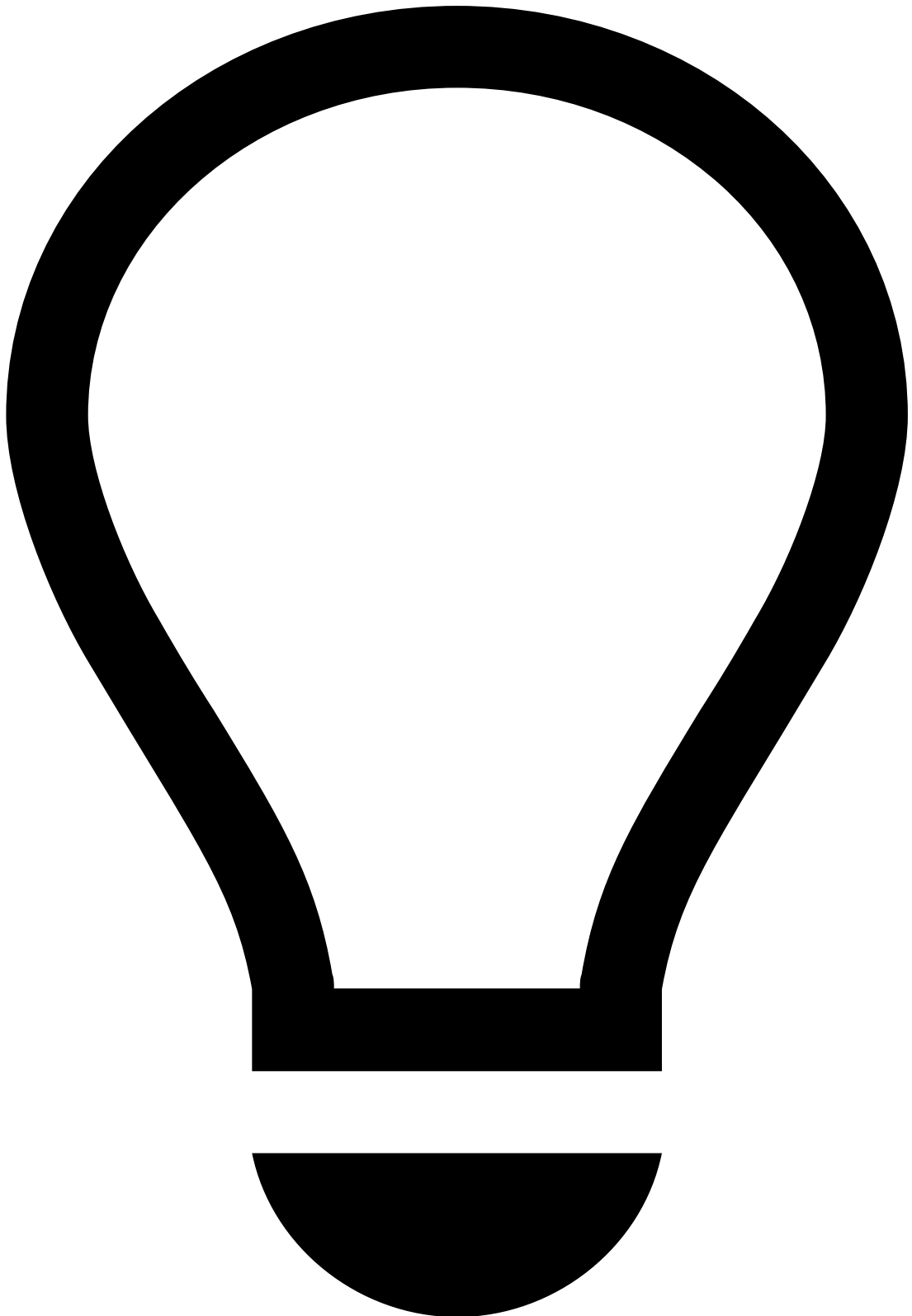
Pulse provides one default job profile that applies to all DS jobs, but it is recommended that you adapt this default profile to better fit the jobs you'll be executing. The default job profile is, essentially, a starting point to create other profiles.

## What are executors?

Executors represent the **main processing unit** that do the actual work in your job (i.e. reading data, transforming it, caching it, and writing results).

When a job is launched/scheduled, the cluster allocates executors on the task nodes (i.e. machines). These executors borrow resources (i.e. cores and memory) from the task nodes to perform the computations and execute DS jobs.

When sizing the executors, be mindful of this to avoid setting unrealistic bounds on the amount and/or size of your executors.



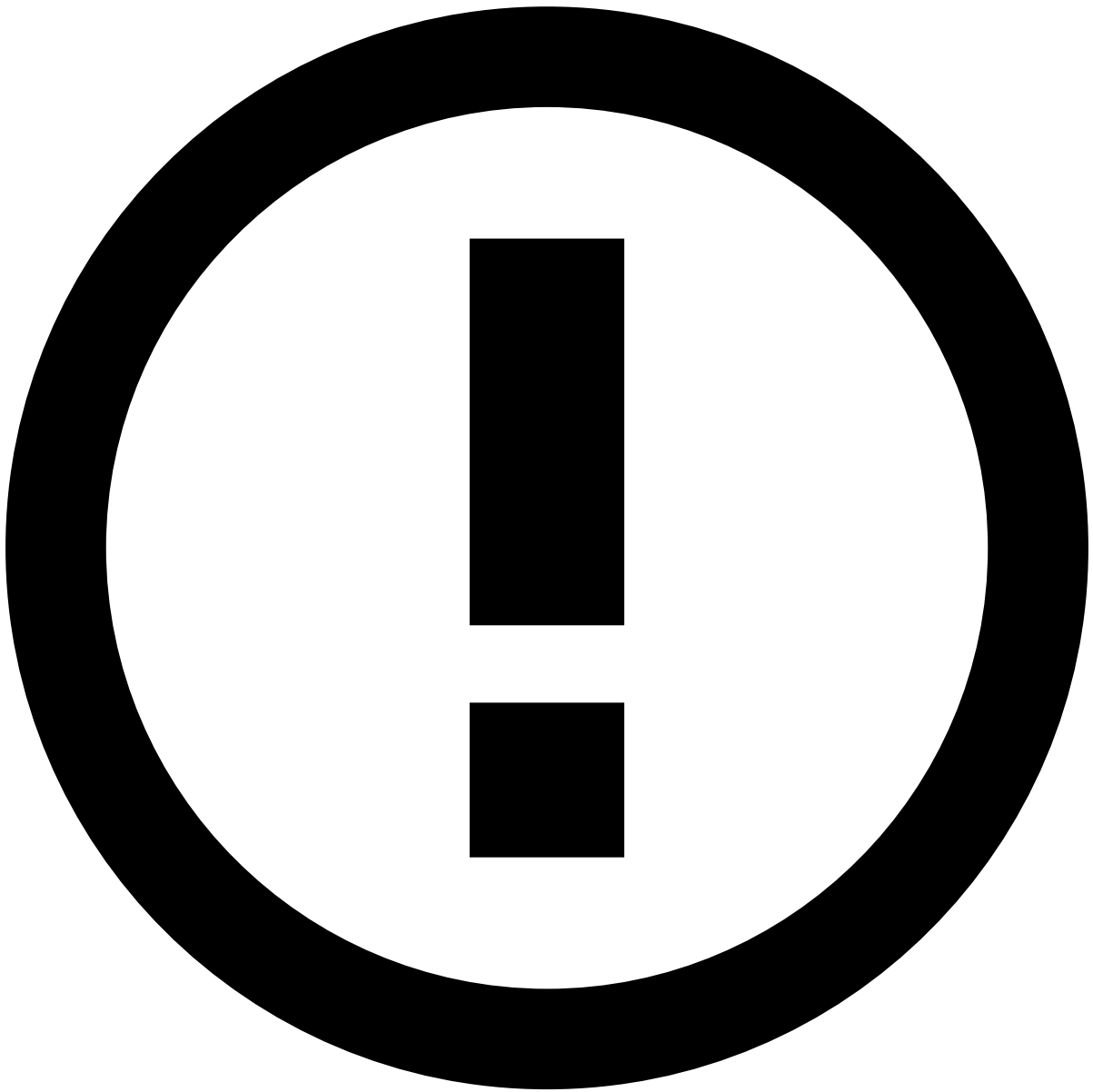
#### Practical example

If you consider a task node with 16 cores and 128GB of memory, configuring executors to have nine cores and 50GB of memory would represent poor cluster resource usage, which would, ultimately, lead to poor performance. Here's why:

After allocating one executor (with nine cores), the cluster is left with seven available. As each executor borrows nine cores, the cluster is not able to allocate any more executors for that job.

This way, it would only be possible to fit one executor, leaving seven cores unutilized.

Note that the remaining seven cores could still be used by other jobs, as long as the executors fit in the task nodes.



#### Dynamic executor allocation

Note that while you can explicitly define how many executors to run, the **general rule-of-thumb favors dynamic allocation of executors**. This lets the cluster decide how many executors to run depending on the workload.

Learn all about the importance and impact of tuning DS jobs in the [Using the Data Science Framework](#) page.